

MODELAGEM DO BANCO — SQUAD 16

Versão 2.0 — Documentação Corrigida e Consolidada

Visão Geral

Este documento descreve a modelagem completa do banco de dados do Simulador Estratégico de Loja. A versão 2.0 corrige as inconsistências identificadas na v1, incluindo: campo de papel (role) no usuário, relação entre RoundConfig e Product, conexão do Inventory ao motor de simulação, rastreabilidade do FinancialResult e remoção do score estático do Squad.

Enum: UserRole

- v1 não possuía enum de papéis. O fluxo de autorização ficava sem base de dados para validar.
- ✓ CORREÇÃO: Adicionado enum UserRole com 3 papéis distintos mapeados ao domínio do sistema.

```
enum UserRole {  
    GAME_MASTER    // Controla rodadas, parâmetros e participantes  
    PLAYER         // Participa da simulação configurando sua loja  
    OBSERVER       // Apenas visualiza – sem permissão de escrita  
}
```

Model: User

- v1 usava leader: Boolean para controle de acesso — insuficiente para 3+ papéis distintos.
- ✓ CORREÇÃO: Substituído por role: UserRole. Campo leader mantido apenas para lógica interna de squad.

```
model User {  
    id          String    @id @default(uuid())  
    name        String  
    email       String    @unique  
    password    String  
    role        UserRole  @default(PLAYER)  
    leader      Boolean    @default(false)  
    squadId     String?  
    createdAt   DateTime  @default(now())  
  
    squad       Squad?    @relation(fields: [squadId], references: [id])  
  
    @@index([squadId])  
}
```

Model: Squad

- v1 tinha score: Float diretamente na entidade Squad. Isso impede histórico por rodada e torna o ranking uma foto estática.

✓ **CORREÇÃO:** Campo score removido. O ranking é calculado on-demand a partir de FinancialResult. Adicionado campo updatedAt para controle de cache se necessário no futuro.

```
model Squad {  
  id          String    @id @default(uuid())  
  name        String  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
  
  users       User[]  
  stores      Store[]  
}
```

Model: Store

Sem alterações estruturais. Mantida a relação 1:1 com Squad.

```
model Store {  
  id          String    @id @default(uuid())  
  name        String  
  initialCapital Float  
  squadId     String  
  createdAt   DateTime  @default(now())  
  
  squad       Squad      @relation(fields: [squadId], references: [id])  
  inventory    Inventory[]  
  roundConfigs RoundConfig[]  
  financialResults FinancialResult[]  
  
  @@index([squadId])  
}
```

Model: Product

Produto global do sistema. O precoCompra é fonte de verdade para o cálculo de custos no motor financeiro.

```
model Product {  
  id          String    @id @default(uuid())  
  name        String  
  purchasePrice Float    // precoCompra – usado em custos = purchasePrice * salesVolume  
  salePrice   Float     // precoVenda padrão – pode ser sobrescrito em RoundConfigItem  
  createdAt   DateTime  @default(now())  
  
  inventory    Inventory[]  
  roundConfigItems RoundConfigItem[]  
}
```

```
}
```

Model: Inventory

■ v1 definia Inventory mas ele não era referenciado por RoundConfig nem pelo motor financeiro. Estoque existia no banco mas era invisível para a simulação.

✓ CORREÇÃO: Inventory agora é consultado pelo SimulationService antes do cálculo. Se salesVolume > quantity, o volume efetivo é limitado pelo estoque disponível.

```
model Inventory {  
    id          String    @id @default(uuid())  
    storeId     String  
    productId   String  
    quantity    Int  
    agingCategory String? // 'fresh', 'aging', 'expired' – para lógica futura de perecíveis  
  
    store      Store    @relation(fields: [storeId], references: [id])  
    product    Product  @relation(fields: [productId], references: [id])  
  
    @@unique([storeId, productId])  
    @@index([storeId])  
}
```

Model: Round

Sem alterações estruturais. Representa cada ciclo da simulação.

```
model Round {  
    id          String    @id @default(uuid())  
    number      Int  
    startDate   DateTime  
    endDate     DateTime  
    status      RoundStatus @default(OPEN)  
  
    roundConfigs RoundConfig[]  
    financialResults FinancialResult[]  
}  
  
enum RoundStatus {  
    OPEN          // Squads podem submeter configurações  
    PROCESSING    // Motor de simulação em execução  
    CLOSED        // Resultados disponíveis, ranking atualizado  
}
```

Model: RoundConfig

■ v1 tinha salePrice e salesVolume diretamente em RoundConfig, sem FK para Product. Isso tornava impossível calcular custos = purchasePrice * volume pois purchasePrice vive em Product.

✓ CORREÇÃO: RoundConfig agora é um cabeçalho. Os itens por produto ficam em RoundConfigItem. Isso suporta múltiplos produtos por rodada e mantém a rastreabilidade.

```
model RoundConfig {
    id          String    @id @default(uuid())
    roundId     String
    storeId     String
    fixedExpenses Float    // Despesas fixas da loja na rodada
    variableExpenses Float // Despesas variáveis da loja na rodada
    submittedAt DateTime @default(now())

    round      Round    @relation(fields: [roundId], references: [id])
    store      Store    @relation(fields: [storeId], references: [id])
    items      RoundConfigItem[]
    financialResult FinancialResult?

    @@unique([roundId, storeId]) // Uma config por loja por rodada
    @@index([roundId, storeId])
}
```

Model: RoundConfigItem [NOVO]

✓ CORREÇÃO: Entidade criada para resolver a ausência de relação entre configuração de rodada e produto. Cada item representa a decisão do squad para um produto específico naquela rodada.

```
model RoundConfigItem {
    id          String    @id @default(uuid())
    roundConfigId String
    productId    String
    salePrice    Float    // Preço de venda definido pelo squad (pode diferir do Product.salePrice)
    salesVolume  Int      // Volume de vendas planejado

    roundConfig RoundConfig @relation(fields: [roundConfigId], references: [id])
    product      Product    @relation(fields: [productId], references: [id])

    @@unique([roundConfigId, productId])
}
```

Model: FinancialResult

■ v1 não tinha FK para RoundConfig. Impossível rastrear quais parâmetros geraram o resultado, inviabilizando auditoria, replay e a funcionalidade 'simular sem salvar'.

✓ CORREÇÃO: Adicionado roundConfigId como FK. FinancialResult agora é o snapshot imutável do resultado de uma RoundConfig específica.

```
model FinancialResult {
  id          String    @id @default(uuid())
  roundId     String
  storeId     String
  roundConfigId String  @unique // 1:1 com RoundConfig – snapshot imutável
  grossRevenue Float    // receitaBruta = sum(salePrice * effectiveSalesVolume) por item
  costs       Float    // custos = sum(purchasePrice * effectiveSalesVolume) por item
  expenses    Float    // despesasFixas + despesasVariaveis
  grossProfit  Float    // lucroBruto = grossRevenue - costs
  netProfit    Float    // lucroLiquido = grossProfit - expenses
  netMargin    Float    // margemLiquida = (netProfit / grossRevenue) * 100
  calculatedAt DateTime @default(now())

  round      Round      @relation(fields: [roundId], references: [id])
  store      Store      @relation(fields: [storeId], references: [id])
  roundConfig RoundConfig @relation(fields: [roundConfigId], references: [id])

  @@index([roundId, storeId])
}
```

Mapa de Relações

Entidade	Relacionamento	Entidade	Cardinalidade
User	belongs to	Squad	N:1
Squad	has	Store	1:1
Store	has	Inventory	1:N
Store	has	RoundConfig	1:N
Store	has	FinancialResult	1:N
Product	has	Inventory	1:N
Product	has	RoundConfigItem	1:N
Round	has	RoundConfig	1:N
Round	has	FinancialResult	1:N
RoundConfig	has	RoundConfigItem	1:N
RoundConfig	has	FinancialResult	1:1